Kalman-Filtered Trend Trader with Reinforcement Learning–Based Rebalancing Control

A Comprehensive Theoretical and Applied Study

Winter in Data Science (WiDS) Final Report

Shaurya Gupta (24B0051)

January 30, 2026

Contents

1 Introduction

Financial markets are complex, stochastic systems influenced by a wide range of economic, psychological, and structural factors. Unlike physical systems, markets do not obey fixed governing laws, and their statistical properties evolve over time. As a result, designing robust quantitative strategies requires careful modeling of uncertainty, noise, and decision-making under incomplete information.

A common pitfall in applying modern data science techniques to finance is the uncritical use of predictive models without accounting for market frictions such as transaction costs, delayed execution, and non-stationarity. Strategies that appear profitable under idealized assumptions often fail when evaluated under realistic conditions.

The objective of this project is not to construct a trading system that maximizes raw returns, but to develop a **principled framework** for portfolio management that explicitly incorporates:

- Noisy observations of market prices
- Latent (unobserved) market dynamics
- Sequential decision-making under uncertainty
- Transaction costs and execution constraints

This work was developed as part of the Winter in Data Science (WiDS) program and represents a synthesis of the concepts covered throughout the program. The final outcome is a portfolio management system that combines **state-space modeling** via Kalman filtering with **reinforcement learning** for cost-aware rebalancing control.

This report is written to be self-contained. A reader unfamiliar with the WiDS curriculum or prior financial modeling experience should be able to understand all concepts used in the final project by progressing through this document.

2 Mathematical Preliminaries

This section introduces the mathematical concepts that form the foundation of the methods used in this project. While these ideas may appear elementary, they are essential for understanding both the modeling assumptions and the limitations of the final system.

2.1 Random Variables and Probability Spaces

A random variable is a function that maps outcomes of a random experiment to real numbers. Formally, given a probability space $(\Omega, \mathcal{F}, \mathbb{P})$, a random variable X is defined as:

$$X : \Omega \to \mathbb{R}$$

In financial modeling, asset prices and returns are treated as random variables due to uncertainty in market behavior. The probabilistic framework allows us to reason about expected outcomes, risk, and variability.

2.2 Expectation

The expectation of a random variable represents its long-run average value. For a discrete random variable:

$$\mathbb{E}[X] = \sum_{x} x \cdot \mathbb{P}(X = x)$$

For a continuous random variable with probability density function $f(x)$:

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x f(x)\, dx$$

In finance, expected returns are often used as a first-order measure of attractiveness, although they are insufficient on their own to fully characterize risk.

2.3 Variance and Risk

Variance measures the dispersion of a random variable around its expected value:

$$\mathrm{Var}(X) = \mathbb{E}\left[(X - \mathbb{E}[X])^2\right]$$

High variance indicates a wide range of possible outcomes and is commonly interpreted as risk in financial contexts. However, variance penalizes both positive and negative deviations equally, which motivates more nuanced risk measures in advanced models.

2.4 Covariance and Correlation

Given two random variables X and Y, covariance is defined as:

$$\mathrm{Cov}(X, Y) = \mathbb{E}\left[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])\right]$$

Covariance plays a central role in portfolio construction, as it determines how asset returns move relative to one another. Low or negative covariance between assets enables diversification, which can reduce overall portfolio risk.

2.5 Time Indexing and Discrete-Time Models

Financial data is typically observed at discrete time intervals (e.g., daily closing prices). Throughout this report, time is indexed discretely as $t = 0, 1, 2, \ldots$.

All models and algorithms in this project are formulated in discrete time to match the structure of real-world market data and to enable practical implementation and backtesting.

2.6 Why Mathematical Foundations Matter

Many failures of quantitative strategies arise not from incorrect implementation, but from misunderstanding the assumptions underlying the mathematical models. By grounding this project in clear probabilistic and statistical principles, we aim to build systems that are:

- Interpretable
- Diagnosable when they fail
- Extendable to more complex settings

These foundations will be repeatedly referenced in later sections, particularly when discussing state-space models, reinforcement learning, and portfolio optimization under constraints.

3 Financial Time Series Theory

Financial time series differ fundamentally from the datasets typically encountered in classical machine learning problems. Understanding these differences is critical before applying any modeling or learning technique to market data.

This section introduces the key statistical properties of financial time series and explains how these properties influence model selection, evaluation, and interpretation throughout this project.

3.1 Prices vs Returns

Let P_t denote the price of an asset at time t. A naïve approach to modeling financial markets is to directly model the price series $\{P_t\}$. However, price series exhibit strong non-stationarity, making them unsuitable for most statistical models.

Instead, financial modeling is typically performed on **returns**, defined as:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

Returns are preferred because:

- They are scale-invariant
- They are closer to stationary
- They allow comparison across assets

Throughout this project, returns are used for performance evaluation, while prices are used for state estimation.

3.2 Stationarity and Non-Stationarity

A time series is said to be **stationary** if its statistical properties (mean, variance, autocorrelation) do not change over time.

Formally, a stochastic process $\{X_t\}$ is weakly stationary if:

$$\begin{aligned} \mathbb{E}[X_t] &= \mu \\ \mathrm{Var}(X_t) &= \sigma^2 \\ \mathrm{Cov}(X_t, X_{t+k}) &= \gamma(k) \end{aligned}$$

for all t and lags k.

Financial price series are generally **non-stationary**, meaning:

- Their mean can drift over time
- Variance can change across regimes
- Past behavior may not generalize to the future

This non-stationarity is a primary reason why overfitted predictive models fail in finance.

3.3 Noise and Low Signal-to-Noise Ratio

One of the defining characteristics of financial markets is their extremely low signal-to-noise ratio. Observed prices contain a large amount of random fluctuation relative to any underlying directional signal.

Mathematically, observed prices can be viewed as:

$$P_t = S_t + \epsilon_t$$

where:

- S_t is the latent signal
- ϵ_t is observation noise

In most liquid markets, the variance of ϵ_t dominates the variance of S_t. This motivates the use of filtering techniques, such as Kalman filters, which explicitly model noise.

3.4 Stylized Facts of Financial Returns

Empirical studies have identified several recurring statistical patterns in financial returns, often referred to as **stylized facts**:

- Heavy-tailed return distributions
- Volatility clustering
- Weak linear autocorrelation
- Strong nonlinear dependencies

These properties violate Gaussian assumptions and limit the effectiveness of simple linear prediction models.

3.5 Autocorrelation and Predictability

Autocorrelation measures the linear dependence between values of a time series at different lags:

$$\rho(k) = \frac{\mathrm{Cov}(r_t, r_{t-k})}{\mathrm{Var}(r_t)}$$

Empirically, daily asset returns exhibit near-zero linear autocorrelation, implying limited predictability at short horizons. However, higher-order structures such as momentum and mean-reversion may still exist at longer horizons.

This distinction motivates trend estimation rather than short-term return prediction.

3.6 Volatility Clustering

While returns themselves may exhibit weak autocorrelation, their magnitude often displays strong persistence. This phenomenon is known as **volatility clustering**.

Periods of high volatility tend to be followed by high volatility, and vice versa. This has important implications for:

- Risk management
- Position sizing
- Portfolio drawdowns

Although volatility modeling is not the primary focus of this project, its effects are indirectly captured through transaction costs and drawdown analysis.

3.7 Implications for Model Design

The properties discussed above impose several constraints on viable modeling approaches:

- Models must adapt to non-stationarity
- Noise must be explicitly accounted for
- Evaluation must be strictly causal
- Over-parameterization should be avoided

These considerations strongly motivate the choice of:

- State-space models for signal extraction
- Reinforcement learning for sequential decision-making
- Cost-aware objectives over raw return maximization

The next section introduces optimization principles that formalize these ideas mathematically.

4 Optimization Theory and Objective Design

At the core of every learning or decision-making system lies an optimization problem. Whether explicitly stated or not, every algorithm attempts to maximize or minimize a well-defined objective function.

In quantitative finance, poor performance often arises not from incorrect algorithms, but from poorly specified objectives that ignore real-world constraints. This section develops the optimization principles that motivate the reward design used later in the reinforcement learning framework.

4.1 Optimization Problems in Finance

A generic optimization problem can be written as:

$$\max_{x} \; f(x)$$

where x represents a decision variable and $f(x)$ is an objective function.

In finance, x often represents:

- Portfolio weights
- Trading actions
- Rebalancing decisions

and $f(x)$ typically represents:

- Expected return
- Risk-adjusted return
- Utility of wealth

However, real financial problems are rarely unconstrained.

4.2 Constraints in Financial Systems

Practical portfolio management is subject to numerous constraints:

- Limited capital
- Transaction costs

- Market impact
- Execution delays

These constraints fundamentally alter the nature of the optimization problem. Ignoring them leads to strategies that appear profitable in theory but fail in practice.

Mathematically, constrained optimization problems can be written as:

$$\max_x \ f(x) \quad \text{subject to} \quad g_i(x) \leq c_i$$

where $g_i(x)$ represents a constraint.

4.3 Transaction Costs as Optimization Constraints

Transaction costs introduce path dependence into portfolio optimization. Let w_t denote portfolio weights at time t. A common proxy for transaction costs is portfolio turnover:

$$\text{Turnover}_t = \sum_i |w_{i,t} - w_{i,t-1}|$$

Transaction costs can be modeled as:

$$\text{Cost}_t = \kappa \cdot \text{Turnover}_t$$

where κ is the per-unit transaction cost rate.

Unlike returns, transaction costs accumulate deterministically and can dominate performance when rebalancing is frequent.

4.4 Lagrangian Relaxation of Constraints

One standard technique for handling constraints is **Lagrangian relaxation**. Rather than enforcing a hard constraint, the constraint is incorporated into the objective using a penalty term.

Given the constrained problem:

$$\max f(x) \quad \text{s.t.} \quad g(x) \leq c$$

we define the Lagrangian:

$$\mathcal{L}(x, \lambda) = f(x) - \lambda(g(x) - c)$$

where $\lambda \geq 0$ is the Lagrange multiplier.

This transforms a constrained problem into an unconstrained one, with λ controlling the tradeoff between objective maximization and constraint satisfaction.

4.5 Application to Portfolio Management

In the context of portfolio management, the unconstrained objective might be:

$$\max \ \mathbb{E}[R_t]$$

where R_t is portfolio return.

Including transaction costs via Lagrangian relaxation yields:

$$\max \ \mathbb{E}[R_t] - \lambda \cdot \mathbb{E}[\text{Turnover}_t]$$

This formulation directly motivates the reward function used in the final project:

$$R_t^{\text{reward}} = \text{Portfolio Return}_t - \lambda \cdot \text{Transaction Cost}_t$$

Thus, the reinforcement learning agent is implicitly solving a constrained optimization problem.

4.6 Why Return Maximization Alone Fails

Maximizing raw returns without penalties leads to pathological behavior:

- Excessive trading
- Sensitivity to noise
- Unrealistic turnover

Empirical results from the final project show that even a transaction cost of 0.1% can erase a significant fraction of gross returns. This observation motivates cost-aware control rather than naive return maximization.

4.7 Risk, Variance, and Objective Design

Although expected return is a natural objective, it does not capture risk. A common extension is mean–variance optimization:

$$\max \ \mathbb{E}[R] - \gamma \cdot \mathrm{Var}(R)$$

While variance-based objectives are theoretically appealing, they can be difficult to estimate reliably in non-stationary environments.

In this project, risk is instead evaluated ex-post using drawdowns and Sharpe ratio, while the learning objective focuses on cost-aware decision-making.

4.8 Implications for Reinforcement Learning

Reinforcement learning algorithms implicitly optimize cumulative reward:

$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R_t\right]$$

By carefully designing R_t to reflect real-world constraints, RL becomes a tool for solving structured optimization problems rather than a black-box predictor.

This perspective strongly influences the design of the RL agent in the final system, where simplicity, interpretability, and stability are prioritized.

4.9 Summary

This section established that:

- Financial decision-making is fundamentally a constrained optimization problem
- Transaction costs must be incorporated directly into the objective
- Lagrangian relaxation provides a principled way to do so
- Reinforcement learning can be interpreted as a numerical optimization method

These ideas form the mathematical bridge between classical optimization theory and the reinforcement learning framework used later in this report.

5 Markov Decision Processes and Dynamic Programming

Many real-world decision-making problems involve making a sequence of decisions over time under uncertainty. The outcome of each decision affects not only the immediate reward, but also the future states of the system.

Portfolio management is a canonical example of such a problem: decisions made today influence capital allocation, transaction costs, and risk exposure in the future. This section introduces the mathematical framework of **Markov Decision Processes (MDPs)**, which provides a rigorous foundation for modeling such problems.

5.1 Sequential Decision-Making Under Uncertainty

Consider an agent interacting with an environment over discrete time steps $t = 0, 1, 2, \ldots$. At each time step:

- The environment is in a state s_t
- The agent selects an action a_t
- The environment transitions to a new state s_{t+1}
- The agent receives a reward R_t

The agent's objective is to choose actions that maximize cumulative reward over time, despite uncertainty in state transitions and rewards.

5.2 Formal Definition of a Markov Decision Process

A Markov Decision Process is defined by the tuple:

$$(S, A, P, R, \gamma)$$

where:

- S is the state space
- A is the action space
- $P(s' \mid s, a)$ is the transition probability

- $R(s, a)$ is the reward function
- $\gamma \in [0, 1)$ is the discount factor

The **Markov property** assumes that the future state depends only on the current state and action, not on the full history:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t)$$

This assumption greatly simplifies analysis and computation.

5.3 Policy and Value Functions

A policy π is a mapping from states to actions:

$$\pi : S \to A$$

The quality of a policy is measured using value functions.

The **state-value function** is defined as:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid s_0 = s \right]$$

The **action-value function** is defined as:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid s_0 = s, a_0 = a \right]$$

These functions quantify the long-term desirability of states and actions.

5.4 Bellman Expectation Equations

The value functions satisfy recursive relationships known as **Bellman equations**.

For a fixed policy π, the Bellman expectation equation for V^π is:

$$V^\pi(s) = \sum_a \pi(a \mid s) \left[R(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^\pi(s') \right]$$

This equation expresses the value of a state as the expected immediate reward plus the discounted value of successor states.

5.5 Bellman Optimality Equations

The optimal value function V^* is defined as:

$$V^*(s) = \max_{\pi} V^{\pi}(s)$$

It satisfies the Bellman optimality equation:

$$V^*(s) = \max_{a} \left[R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) V^*(s') \right]$$

Similarly, the optimal action-value function satisfies:

$$Q^*(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \max_{a'} Q^*(s',a')$$

These equations form the theoretical basis for reinforcement learning algorithms.

5.6 Dynamic Programming

Dynamic programming refers to a class of algorithms that solve MDPs by iteratively applying Bellman equations.

Two canonical dynamic programming methods are:

- Value iteration
- Policy iteration

Both methods rely on full knowledge of the transition probabilities $P(s'|s,a)$, which is rarely available in real-world financial problems.

5.7 Curse of Dimensionality

A major challenge in applying dynamic programming is the **curse of dimensionality**. As the size of the state space grows, exact computation of value functions becomes infeasible.

This motivates:

- State discretization
- Approximate methods
- Reinforcement learning algorithms

In this project, careful state design is used to keep the problem tractable without sacrificing interpretability.

5.8 Why Portfolio Rebalancing Fits the MDP Framework

Portfolio management naturally fits the MDP framework:

- States summarize market conditions and portfolio status
- Actions correspond to rebalancing decisions
- Rewards capture returns and transaction costs
- Future states depend on both market dynamics and actions

Crucially, rebalancing decisions affect future transaction costs, making the problem inherently sequential.

5.9 Interpretation for This Project

In the final system:

- States encode aggregate trend information
- Actions are discrete: hold or rebalance
- Rewards incorporate both returns and costs

The RL agent implicitly approximates the solution to the Bellman optimality equation, without requiring explicit knowledge of transition probabilities.

5.10 Summary

This section established that:

- Portfolio management is a sequential decision problem
- MDPs provide a rigorous modeling framework
- Bellman equations define optimal behavior
- Reinforcement learning offers a practical solution when dynamics are unknown

The next section builds on this foundation by introducing reinforcement learning algorithms that learn optimal policies directly from data.

6 Reinforcement Learning Theory

Reinforcement learning (RL) provides a framework for solving Markov Decision Processes when the transition dynamics of the environment are unknown. Instead of relying on explicit models of state transitions, an RL agent learns optimal behavior directly from interaction with the environment.

In financial settings, transition probabilities are rarely known or stationary, making reinforcement learning a natural candidate for sequential decision-making. However, care must be taken to design RL systems that are stable, interpretable, and robust to noise.

6.1 Model-Based vs Model-Free Approaches

Reinforcement learning algorithms can be broadly categorized into:

- **Model-based methods**, which attempt to learn $P(s'|s, a)$
- **Model-free methods**, which learn value functions directly

Model-based methods can be sample-efficient but are sensitive to model misspecification. In contrast, model-free methods avoid explicit modeling assumptions and are often more robust in non-stationary environments.

This project adopts a model-free approach.

6.2 Value-Based Reinforcement Learning

Value-based RL methods focus on learning value functions that quantify the desirability of states or state-action pairs.

The most commonly used value function is the action-value function:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t R_t \mid s_0 = s, a_0 = a\right]$$

An optimal policy can be obtained by acting greedily with respect to Q^*:

$$\pi^*(s) = \arg\max_a Q^*(s, a)$$

6.3 Q-Learning Algorithm

Q-learning is an off-policy, model-free RL algorithm that estimates the optimal action-value function $Q^*(s, a)$.

The update rule is given by:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(R_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right)$$

where:

- α is the learning rate
- γ is the discount factor

The term inside parentheses is known as the **temporal difference (TD) error**.

6.4 Temporal Difference Learning

Temporal difference learning combines ideas from dynamic programming and Monte Carlo methods. It updates value estimates based on other learned estimates rather than waiting for full episode termination.

The TD error is defined as:

$$\delta_t = R_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)$$

A positive TD error indicates that the outcome was better than expected, while a negative TD error indicates underperformance.

6.5 Exploration vs Exploitation

A fundamental challenge in reinforcement learning is the exploration–exploitation tradeoff.

- **Exploitation**: choosing actions believed to be optimal
- **Exploration**: trying alternative actions to improve estimates

This project employs an ϵ-greedy exploration strategy:

$$a_t = \begin{cases} \text{random action}, & \text{with probability } \epsilon \\ \arg\max_a Q(s_t, a), & \text{with probability } 1 - \epsilon \end{cases}$$

The exploration rate ϵ is decayed over time to favor exploitation as learning progresses.

6.6 Convergence Properties of Q-Learning

Under the following conditions:

- Finite state and action spaces
- Sufficient exploration
- Decaying learning rate

Q-learning is guaranteed to converge to the optimal action-value function Q^*.

These theoretical guarantees strongly motivate the use of tabular Q-learning in this project, where state discretization keeps the problem finite and tractable.

6.7 Stability Considerations in Financial Applications

Financial environments pose additional challenges for RL:

- High noise levels
- Non-stationary dynamics
- Sparse and delayed rewards

Complex function approximators, such as deep neural networks, can amplify these issues and lead to unstable learning. For this reason, this project deliberately avoids deep reinforcement learning in favor of simpler, more interpretable methods.

6.8 Why Tabular Q-Learning is Appropriate Here

The decision problem addressed in this project has:

- A low-dimensional state representation
- A small discrete action space
- Strong structural constraints

Tabular Q-learning offers several advantages:

- Exact value representation
- Transparent learning dynamics
- Ease of debugging and interpretation

These properties make it well-suited for cost-aware portfolio control.

6.9 Interpretation of RL as Optimization

Reinforcement learning can be viewed as an iterative method for solving the Bellman optimality equation. Each Q-learning update moves the estimate closer to the fixed point defined by the Bellman operator.

From this perspective, RL is not a black-box heuristic, but a principled numerical optimization technique.

6.10 Summary

This section established that:

- Reinforcement learning solves MDPs without explicit models
- Q-learning estimates optimal action-value functions
- Exploration is essential for convergence
- Simplicity and structure improve stability in finance

The next section introduces state-space models and Kalman filtering, which provide the estimation layer used by the RL-controlled portfolio system.

7 State-Space Models and Kalman Filter Theory

Many real-world systems evolve according to underlying dynamics that are not directly observable. Instead, we observe noisy measurements that provide partial information about the system state. State-space models provide a principled mathematical framework for modeling such systems.

In financial markets, observed prices are noisy manifestations of latent economic forces such as trend, momentum, and regime. This motivates the use of state-space models for signal extraction.

7.1 Latent Variable Modeling

A latent variable model assumes the existence of hidden variables that govern observed data. Let:

- x_t denote the latent (hidden) state at time t
- y_t denote the observed variable

The goal is to infer x_t given observations $\{y_1, y_2, \ldots, y_t\}$.

In finance, x_t may represent trend or momentum, while y_t corresponds to observed prices.

7.2 General State-Space Model

A discrete-time linear state-space model is defined by two equations:

State Transition Equation

$$x_{t+1} = Fx_t + w_t$$

Observation Equation

$$y_t = Hx_t + v_t$$

where:

- F is the state transition matrix
- H is the observation matrix
- $w_t \sim \mathcal{N}(0, Q)$ is process noise
- $v_t \sim \mathcal{N}(0, R)$ is observation noise

The noise terms model uncertainty in system dynamics and measurements.

7.3 Interpretation of Process and Observation Noise

The process noise covariance Q captures uncertainty in the evolution of the latent state. A larger Q allows the state to change more rapidly, while a smaller Q enforces smoother dynamics.

The observation noise covariance R captures trust in the observed data. A larger R implies noisier measurements and places more weight on the model prediction.

The relative magnitudes of Q and R determine the bias–variance tradeoff of the filter.

7.4 Filtering Problem

The filtering problem consists of computing the posterior distribution:

$$p(x_t \mid y_1, y_2, \ldots, y_t)$$

For linear systems with Gaussian noise, this posterior is also Gaussian. The Kalman filter provides a recursive solution for computing its mean and covariance.

7.5 Kalman Filter Assumptions

The Kalman filter relies on the following assumptions:

- Linear system dynamics
- Gaussian process and observation noise
- Known model parameters (F, H, Q, R)

While these assumptions are idealized, the filter often performs well even when they are only approximately satisfied.

7.6 Kalman Filter: Prediction Step

Given the posterior at time t:

$$x_t \sim \mathcal{N}(\hat{x}_t, P_t)$$

the predicted state at time $t+1$ is:

$$\hat{x}_{t+1|t} = F\hat{x}_t$$

The predicted covariance is:

$$P_{t+1|t} = FP_tF^\top + Q$$

This step propagates uncertainty forward in time.

7.7 Kalman Filter: Update Step

Upon observing y_{t+1}, the filter updates its belief:

Innovation

$$\tilde{y}_{t+1} = y_{t+1} - H\hat{x}_{t+1|t}$$

Innovation Covariance

$$S_{t+1} = HP_{t+1|t}H^\top + R$$

Kalman Gain

$$K_{t+1} = P_{t+1|t}H^\top S_{t+1}^{-1}$$

State Update

$$\hat{x}_{t+1} = \hat{x}_{t+1|t} + K_{t+1}\tilde{y}_{t+1}$$

Covariance Update

$$P_{t+1} = (I - K_{t+1}H)P_{t+1|t}$$

The Kalman gain determines how much the estimate is corrected based on new information.

7.8 Optimality of the Kalman Filter

The Kalman filter is optimal in the sense that it minimizes the mean squared error among all linear unbiased estimators.

This optimality result explains why the Kalman filter remains a fundamental tool in engineering, control systems, and quantitative finance.

7.9 Application to Trend Estimation

In this project, the latent state is defined as:

$$x_t = \begin{bmatrix} \text{Price}_t \\ \text{Trend}_t \end{bmatrix}$$

with state transition matrix:

$$F = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

This structure assumes that:

- Price evolves as the integral of trend
- Trend evolves slowly over time

The Kalman filter thus produces a smooth estimate of trend that adapts to market changes.

7.10 Why Kalman Filters are Suitable for Finance

Kalman filters offer several advantages in financial applications:

- Explicit noise modeling

- Recursive, online updates
- Interpretable state estimates
- Graceful degradation under model mismatch

These properties make them preferable to ad-hoc smoothing techniques such as moving averages.

7.11 Limitations

Despite their strengths, Kalman filters have limitations:

- Linear dynamics assumption
- Gaussian noise assumption
- Sensitivity to parameter selection

These limitations motivate cautious interpretation and robust evaluation.

7.12 Summary

This section established that:

- State-space models separate latent dynamics from noisy observations
- Kalman filters provide optimal recursive estimation under Gaussian assumptions
- Trend estimation can be framed as a filtering problem
- Kalman filters form a principled signal extraction layer for portfolio systems

The next section builds upon this estimation framework to develop portfolio construction and transaction cost modeling.

8 Portfolio Theory, Allocation Rules, and Transaction Costs

Portfolio management is the process of allocating capital across a set of assets in order to achieve a desired balance between return and risk. While much of classical portfolio

theory focuses on static allocation, real-world portfolio management is inherently dynamic and path-dependent.

This section develops the principles governing portfolio construction in this project, with particular emphasis on allocation rules, cash management, and transaction costs.

8.1 Portfolio Weights and Capital Constraints

Let $w_{i,t}$ denote the fraction of total portfolio capital allocated to asset i at time t. Portfolio weights satisfy the constraint:

$$\sum_i w_{i,t} \leq 1$$

Any unallocated capital is held as cash. This formulation allows the portfolio to reduce risk exposure when market signals are weak, rather than forcing full investment at all times.

8.2 Role of Cash as an Asset

In many simplified models, cash is ignored or implicitly assumed to be fully invested. However, in real trading systems, cash plays a crucial role:

- It limits downside risk
- It reduces transaction costs
- It provides flexibility under uncertainty

In this project, cash is treated as an explicit allocation outcome rather than an afterthought.

8.3 Signal-to-Weight Mapping

The Kalman filter produces a continuous estimate of trend for each asset. These trend estimates must be transformed into portfolio weights.

The mapping used in this project is:

$$\tilde{w}_{i,t} = \max(\text{Trend}_{i,t}, 0)$$

Negative trends are clipped to zero to avoid short positions. The raw weights are then normalized:

$$w_{i,t} = \frac{\tilde{w}_{i,t}}{\sum_j \tilde{w}_{j,t}}$$

If all trends are non-positive, the portfolio remains fully in cash.

8.4 Interpretability of the Allocation Rule

The chosen allocation rule has several desirable properties:

- Monotonicity: stronger trends receive larger weights
- Interpretability: weights directly reflect signal strength
- Stability: avoids extreme leverage or oscillations

This simplicity is intentional and supports robustness.

8.5 Dynamic Rebalancing

As trends evolve over time, target portfolio weights change. Rebalancing refers to adjusting holdings to match new target weights.

Frequent rebalancing improves responsiveness but increases transaction costs. Infrequent rebalancing reduces costs but risks holding stale positions.

This tradeoff motivates the use of reinforcement learning for rebalancing control.

8.6 Transaction Cost Modeling

Transaction costs are modeled as a linear function of portfolio turnover. Let V_t denote total portfolio value. The dollar turnover at time t is:

$$\text{Turnover}_t = \sum_i |w_{i,t} - w_{i,t-1}| \cdot V_t$$

Transaction cost is then given by:

$$\text{Cost}_t = \kappa \cdot \text{Turnover}_t$$

where κ is the per-unit transaction cost rate.

8.7 Path Dependence and Cost Accumulation

Unlike returns, transaction costs accumulate deterministically. Two strategies with identical gross returns can have vastly different net performance depending on turnover.

This path dependence means that rebalancing decisions must account for future cost implications, not just immediate returns.

8.8 Impact of Overtrading

Naive trend-following strategies tend to overtrade due to noise-induced signal fluctuations. This leads to:

- High turnover
- Increased drawdowns
- Erosion of gross alpha

Empirical results from this project show that transaction costs of just 0.1% can eliminate a large fraction of gross returns over long horizons.

8.9 Why Transaction Costs Must Enter the Objective

Because transaction costs are incurred as a direct consequence of actions, they must be included in the optimization objective rather than treated as an afterthought.

This insight directly motivates the cost-aware reward function used in the RL framework.

8.10 Interaction with Reinforcement Learning

In the final system:

- Portfolio weights are determined deterministically by signals
- RL controls the timing of rebalancing
- Transaction costs influence reward

This separation allows RL to focus on the core control problem without learning redundant signal representations.

8.11 Summary

This section established that:

- Portfolio allocation is constrained by capital and costs
- Cash is an important control variable
- Turnover drives transaction cost accumulation
- Cost-aware design is essential for realistic performance

The next section formalizes the causal backtesting framework used to evaluate the system under realistic trading conditions.

9 Causal Backtesting and Evaluation Methodology

Backtesting is the process of evaluating a trading strategy using historical data. While backtesting is essential for understanding strategy behavior, it is also a common source of misleading conclusions when implemented incorrectly.

This section presents the causal backtesting framework used in this project and explains the methodological choices required to obtain realistic performance estimates.

9.1 What is Lookahead Bias

Lookahead bias occurs when a strategy makes decisions using information that would not have been available at the time the decision was made.

Examples include:

- Using same-day closing prices to decide same-day trades
- Computing indicators using future data
- Optimizing parameters on the full dataset

Lookahead bias artificially inflates performance and invalidates conclusions.

9.2 Time Indexing and Information Sets

To avoid lookahead bias, it is necessary to carefully define the information set available at each decision time.

Let:

- $\mathcal{I}_t$ denote all information available up to time t
- Decisions at time t must be functions of $\mathcal{I}_{t-1}$

This distinction is critical when working with daily financial data.

9.3 Signal Lagging

In this project, signals are derived from price data observed up to time $t-1$. Portfolio decisions based on these signals are executed at time t.

Formally:

$$\text{Signal}_t = f(P_1, P_2, \ldots, P_{t-1})$$

This ensures that no future information is leaked into decision-making.

9.4 Execution Lag

In real markets, trades cannot be executed instantaneously at observed prices. To reflect this, all trades in the backtest are executed with a one-period delay.

This models:

- Order placement latency
- Market microstructure effects
- Operational constraints

Execution lag further enforces causality.

9.5 Transaction Cost Application

Transaction costs are applied at the time of execution, not at the time of signal generation.

At each rebalance:

$$\text{Cost}_t = \kappa \cdot \sum_i |w_{i,t} - w_{i,t-1}| \cdot V_t$$

This cost is deducted directly from portfolio value, introducing path dependence into the backtest.

9.6 Mark-to-Market Valuation

Between rebalancing events, portfolio value is updated using mark-to-market pricing:

$$V_t = \text{Cash}_t + \sum_i \text{Holdings}_{i,t} \cdot P_{i,t}$$

This ensures that portfolio performance reflects true exposure to market movements.

9.7 Benchmark Construction

Benchmark selection is a critical but often overlooked aspect of strategy evaluation. Rather than comparing against a single asset, a fair benchmark is constructed as an equal-weight portfolio of the traded assets.

This allows performance comparison against overall market exposure rather than a specific index.

9.8 Performance Metrics

Multiple metrics are required to capture different aspects of performance.

Cumulative Return Measures total growth of capital over the backtest horizon.

Sharpe Ratio Defined as:

$$\text{Sharpe} = \frac{\mathbb{E}[r_t]}{\text{Std}(r_t)}\sqrt{252}$$

This metric captures risk-adjusted return.

Maximum Drawdown Measures the largest peak-to-trough loss:

$$\text{MDD} = \min_t \left(\frac{V_t - \max_{s \le t} V_s}{\max_{s \le t} V_s} \right)$$

Drawdown captures tail risk ignored by variance-based metrics.

9.9 Turnover and Rebalancing Frequency

Turnover measures how frequently the portfolio is traded. High turnover is strongly correlated with transaction cost drag.

Rebalancing frequency is defined as the fraction of time steps in which the portfolio is actively rebalanced.

In this project, rebalancing frequency provides insight into the behavior learned by the RL agent.

9.10 Interpretation of Results

Performance metrics must be interpreted holistically. A strategy with high raw returns but excessive drawdowns or turnover may be unsuitable for deployment.

The results of this project highlight the importance of cost-aware control and demonstrate that naive signal exploitation can be detrimental.

9.11 Summary

This section established that:

- Causality is essential for valid backtesting
- Signal lag and execution delay prevent lookahead bias
- Transaction costs introduce path dependence
- Multiple metrics are needed to evaluate strategy quality

The next section presents the full system architecture and algorithmic flow of the final implementation.

10 Final System Architecture and Algorithmic Flow

The final system developed in this project integrates concepts from state-space modeling, reinforcement learning, optimization, and causal backtesting into a unified portfolio management framework.

A key design principle throughout the project is **separation of concerns**. Rather than using a monolithic model, the system is decomposed into modular components, each responsible for a distinct function.

10.1 High-Level Architecture

The system consists of three primary layers:

1. **Estimation Layer** – Kalman filtering for trend extraction
2. **Decision Layer** – Reinforcement learning for rebalancing control
3. **Execution Layer** – Portfolio updates and transaction cost handling

This layered architecture improves interpretability, stability, and extensibility.

10.2 Estimation Layer: Kalman Trend Tracking

At each time step t, the estimation layer processes observed prices $\{P_{i,t}\}$ for each asset i.

Using the state-space model described earlier, the Kalman filter updates:

- Estimated price
- Estimated trend
- Associated uncertainty

Only information available up to time $t-1$ is used for decision-making, ensuring strict causality.

10.3 Signal Aggregation

Individual asset trends are aggregated into a low-dimensional representation used by the RL agent.

Specifically:

- Aggregate trend strength summarizes overall market opportunity
- Trend change captures regime shifts

This aggregation reduces dimensionality while preserving relevant structure.

10.4 Decision Layer: Reinforcement Learning Controller

The decision layer is responsible solely for determining **whether** to rebalance the portfolio at a given time step.

The RL agent:

- Observes the aggregated state
- Selects an action: hold or rebalance
- Receives reward based on realized performance

Importantly, the agent does not learn portfolio weights, which are handled deterministically by the estimation layer.

10.5 Action Interpretation

The action space is defined as:

- Action 0: Maintain current portfolio allocation
- Action 1: Rebalance portfolio according to target weights

This binary action design dramatically simplifies learning while preserving economic relevance.

10.6 Execution Layer

When a rebalance action is triggered:

1. Target weights are computed from Kalman trends
2. Dollar allocations are derived from portfolio value
3. Trades are executed at current market prices
4. Transaction costs are deducted

When no rebalance occurs, holdings are carried forward unchanged.

10.7 Transaction Cost Accounting

Transaction costs are applied proportionally to portfolio turnover. These costs directly reduce portfolio value and influence future rewards.

This explicit accounting ensures that trading frequency is economically meaningful.

10.8 Time-Step Algorithm

The complete algorithm at each timestep proceeds as follows:

1. Observe prices at time t
2. Update Kalman filters
3. Construct target portfolio weights
4. Compute RL state from lagged signals
5. Select action via Q-learning policy

6. Execute trades if rebalancing is chosen
7. Update portfolio value
8. Compute reward
9. Update Q-table

This loop is repeated sequentially over the entire dataset.

10.9 Training vs Evaluation Phases

During training:

- Exploration is enabled via ϵ-greedy policy
- Q-values are updated after each timestep

During evaluation:

- Exploration is disabled
- Learned policy is executed deterministically

This separation prevents contamination of evaluation results.

10.10 Design Rationale

Key architectural decisions were guided by:

- Stability under noise
- Interpretability of decisions
- Minimal reliance on hyperparameter tuning

The resulting system avoids common pitfalls of deep reinforcement learning while retaining the benefits of sequential optimization.

10.11 Extensibility

The modular structure allows straightforward extensions, including:

- Alternative signal models
- Regime-aware state representations
- Adaptive transaction cost penalties

These extensions can be implemented without redesigning the entire system.

10.12 Summary

This section demonstrated how theoretical components are integrated into a coherent, end-to-end portfolio management system.

The next section analyzes experimental results and interprets system behavior in detail.

11 Experimental Results and Quantitative Analysis

This section presents the empirical performance of the proposed system and analyzes its behavior under realistic trading assumptions. The emphasis is placed not only on aggregate performance metrics, but also on understanding *why* the strategy behaves the way it does.

11.1 Experimental Setup

The backtest was conducted over a nine-year period from 2015 to 2024 using daily price data for the following assets:

- Bitcoin (BTC)
- Nifty 50 index
- Gold (GLD ETF)

An initial capital of $100,000 was used. Transaction costs were modeled at a rate of 0.1% per unit of turnover. All decisions were made using strictly lagged information to ensure causality.

11.2 Baseline Benchmark

To provide a fair comparison, a benchmark portfolio was constructed as an equal-weight allocation across the three traded assets. This benchmark captures broad market exposure and avoids reliance on any single asset class.

11.3 Aggregate Performance Metrics

The following metrics summarize strategy performance:

- Final Portfolio Value: $2.52M
- Benchmark Final Value: $9.87M
- Net Sharpe Ratio: 0.81
- Benchmark Sharpe Ratio: 0.89
- Maximum Drawdown: -69.3%
- Rebalance Frequency: 86%

While the benchmark outperformed in absolute returns, the strategy maintained competitive risk-adjusted performance despite explicit transaction cost penalties.

11.4 Interpretation of Sharpe Ratio

The Sharpe ratio measures excess return per unit of volatility. A Sharpe of 0.81 indicates moderate risk-adjusted performance, especially given the inclusion of high-volatility assets such as Bitcoin.

Importantly, this Sharpe ratio is achieved *net of transaction costs*, highlighting the robustness of the control framework.

11.5 Drawdowns and Tail Risk

The observed maximum drawdown of -69.3% reflects the exposure to highly volatile assets and trend reversals. This underscores the limitations of trend-following strategies during prolonged adverse regimes.

Drawdown analysis highlights the importance of incorporating risk-sensitive objectives in future extensions.

11.6 Turnover and Rebalancing Behavior

The reinforcement learning agent chose to rebalance on approximately 86% of trading days. This high frequency indicates that, under the chosen reward structure, the agent prioritized responsiveness over cost minimization.

Despite this, transaction costs were explicitly accounted for, revealing their dominant role in shaping net performance.

11.7 Cost Attribution Analysis

A key empirical finding is that transaction costs eroded approximately 70% of gross returns over the backtest horizon. This result demonstrates that naive exploitation of noisy signals is economically infeasible at scale.

The analysis motivates the central design decision of this project: optimizing *control decisions* rather than raw signal strength.

11.8 Behavioral Insights from the RL Agent

Inspection of the learned policy reveals that:

- Rebalancing is favored during periods of rapidly changing trends
- Holding is preferred during stable or low-signal regimes

This behavior aligns with economic intuition and validates the MDP formulation.

12 Failure Modes, Limitations, and Negative Results

No quantitative system is complete without a clear understanding of its limitations. This section documents known weaknesses and failure modes observed during experimentation.

12.1 Sensitivity to Transaction Cost Assumptions

The system is highly sensitive to the assumed transaction cost rate. Higher costs significantly degrade performance, while lower costs encourage excessive trading.

This sensitivity highlights the importance of realistic cost modeling.

12.2 Trend Model Limitations

The Kalman filter assumes linear dynamics and Gaussian noise. In practice, financial markets exhibit nonlinear behavior and heavy-tailed distributions.

While the filter performs robustly under noise, it cannot capture sudden structural breaks perfectly.

12.3 Reinforcement Learning Limitations

The RL agent operates on a discretized state space, which limits resolution. Although this improves stability, it may obscure fine-grained market signals.

Additionally, the agent optimizes a myopic reward function and does not explicitly reason about tail risk.

12.4 Underperformance Relative to Benchmark

The strategy underperforms the benchmark in absolute returns. This is not a failure of implementation, but a reflection of conservative, cost-aware design choices.

The goal of this project is understanding and robustness, not aggressive alpha extraction.

13 Learnings from the Winter in Data Science Program

This project represents a synthesis of the core concepts covered throughout the WiDS program.

Key learnings include:

- How optimization objectives shape model behavior
- Why causality is essential in financial modeling
- The dangers of over-parameterization in noisy environments
- The importance of separating estimation from control

Mini-projects completed during the program, including linear regression from scratch, MDP solvers, and tabular reinforcement learning, directly informed design decisions in the final system.

13.1 Conceptual Growth

Perhaps the most important outcome of this work is a shift in perspective: from viewing models as predictors, to viewing them as components in a constrained decision system.

This mindset is essential for real-world quantitative research.

14 Future Work

Several extensions could build upon this framework:

- Incorporating volatility-aware reward functions
- Introducing regime classification in the state space
- Using adaptive transaction cost penalties
- Extending to multi-objective optimization

Each of these extensions can be implemented without altering the core architecture.

15 Conclusion

This report presented a comprehensive study of portfolio management under uncertainty, combining state-space modeling and reinforcement learning within a rigorously causal evaluation framework.

The central insight is that financial decision-making is best approached as a constrained optimization problem, where structure, interpretability, and realism are more valuable than raw model complexity.

By emphasizing estimation, control, and cost-awareness, this project demonstrates a principled approach to quantitative system design that prioritizes understanding over superficial performance.